



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"Place to Learn, Chance to Grow"
2025-2026 (EVEN Semester)



VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Branch: Artificial Intelligence
Subject: LFTO

Semester: 6th Sem 3rd Year

UNIT -IV

Optimization Techniques: Optimizers: Adam, AdamW, Adafactor Gradient clipping, learning rate scheduling, and mixed precision training, Knowledge Distillation and Quantization, Pruning and Compression for deployment efficiency

LLM optimization is hard in production environments, especially when models and applications begin to scale across many teams, tenants, and geographies. Organizations quickly learn that most of the challenge is not training new models from scratch, but running existing large language models efficiently and reliably in production while meeting latency and cost expectations.

As usage grows, the infrastructure that hosts large language models and AI applications becomes a primary driver of success or failure. The way you place models near data, schedule and bin pack GPU workloads, and enforce multi-tenant and sovereignty boundaries has more impact on real-world outcomes than marginal changes in model architecture. This guide focuses on that version of LLM optimization: optimizing how you deploy and operate LLM powered applications on GPUs so that performance, cost, and compliance constraints are all satisfied.

Key highlights:

What LLM optimization means in production. This guide focuses on optimizing how you deploy and operate LLM powered applications on GPUs, improving performance, efficiency, and scalability through better platform and infrastructure decisions rather than model design or training.

Why optimization matters as usage scales. As LLM adoption grows, organizations face mounting pressure on latency, GPU utilization, and cost per token. Many organizations find that inference costs represent a large share of their AI budgets, and platform level optimization can deliver double digit improvements in throughput and cost efficiency.

Practical techniques for production workloads. The article covers quantization, batching, KV cache management, parallelism, efficient attention, and other techniques that connect to GPU selection and platform level controls, all applicable without retraining models from scratch.

Platform level solutions from Mirantis. Mirantis k0rdent AI provides a composable, Kubernetes based platform that helps Neoclouds and enterprises operationalize LLM optimization at scale, maximizing GPU ROI while preserving zero lock in and strong sovereignty.

What Is LLM Optimization?

LLM optimization, in the context of this guide, is the discipline of improving the performance, efficiency, and scalability of large language model powered applications in production. Rather than focusing on how to train new models or design new architectures, LLM optimization here means selecting the right deployment patterns, infrastructure components, and runtime techniques so that inference is fast, predictable, and cost-effective. In practical terms, large language model optimization includes choices about which models to host, how to place those models near the data and storage they need, which GPUs to use, how to keep those GPUs saturated with useful work, and how to enforce multi-tenant and sovereignty boundaries while still achieving high utilization. For Neoclouds and enterprises, this is where most impact will come from: by optimizing model hosting and application architecture, they can reduce per token cost, improve response time, and deliver consistent experience to many tenants without constant manual tuning.



Adam Optimizer

Introduction

In the intricate world of machine learning, optimization algorithms are the engines that drive our models towards better accuracy and performance. They guide how a model adjusts its parameters during training to minimize errors. Among the array of optimizers available, Adam stands out as a true powerhouse, widely favored for its efficiency and adaptability. In this blog post, we'll dive into the heart of the Adam optimizer, unraveling its secrets and shedding light on its uses.

What is Adam?

Adam, short for Adaptive Moment Estimation, is an optimization algorithm that builds upon the strengths of two other popular techniques: AdaGrad and RMSProp. Like its predecessors, Adam is an adaptive learning rate algorithm. This means it dynamically adjusts the learning rate for each individual parameter within a model, rather than using a single global learning rate.

Why Use Adam?

Here's why Adam has become so prevalent in machine learning:

Speed and Efficiency: Adam leverages past gradient information to accelerate convergence. This often leads to faster training times compared to simpler optimizers like basic Stochastic Gradient Descent (SGD).

Handles Sparse Gradients: Datasets where features occur infrequently (sparse gradients) can challenge some optimizers. Adam is designed to be more robust in such scenarios.

Adaptive Learning Rates: The individual adjustments to learning rates make Adam suitable for problems with varying data or parameter landscapes.

Minimal Hyperparameter Tuning: Adam generally performs well with minimal tweaking of the default settings, simplifying the model development process.

How Adam Works

Let's break down the mechanics behind Adam's magic:

Momentum: Adam keeps track of an exponentially decaying average of past gradients (similar to momentum in SGD). This helps to smooth out the updates and navigate noisy gradients.

Adaptive Learning Rates: Adam also computes an exponentially decaying average of past squared gradients. This is used to scale the learning rate for each parameter, allowing for larger updates for infrequent features and smaller updates for frequent ones.

Bias Correction: In the initial iterations, the averages computed by Adam may be biased towards zero. Adam incorporates a bias correction step to counteract this early bias.

When to Use Adam And When Not to

Adam is an excellent starting point for most machine learning problems, particularly in deep learning. Here's when it shines:

Deep Neural Networks: The speed and adaptability of Adam make it perfect for the complex landscapes of deep neural networks.

Computer Vision Tasks: Adam works well with the vast image datasets frequently used in computer vision.

Natural Language Processing: NLP tasks often benefit from the benefits Adam offers.

However, there are scenarios where other optimizers might be better suited:

Small Datasets: With smaller datasets, simpler optimizers like SGD might sometimes converge faster and generalize better.

Need for Precise Control: For problems requiring very fine-tuned control over the learning process, consider SGD with manual learning rate schedules.

AdamW Optimizer

Origins & Motivation

AdamW is a direct successor to the famous **Adam optimizer** (Adaptive Moment Estimation), introduced by Kingma & Ba in 2014. Adam itself was already a huge improvement over SGD, combining momentum (first moment) with adaptive per-parameter learning rates (second moment). However, it had a subtle but significant flaw in how it implemented *L2 regularization*.

In standard Adam, weight decay was incorrectly coupled with the adaptive learning rate, effectively making the regularization weaker for parameters with large gradient variance. **AdamW** (Loshchilov & Hutter, 2017) decoupled weight decay from the gradient update, applying it directly to the weights — which is how true L2 regularization is supposed to work.

How AdamW Works

AdamW maintains two running statistics for each parameter:

- **First moment (m):** An exponentially weighted moving average of gradients — captures the direction of the gradient (similar to momentum).
- **Second moment (v):** An exponentially weighted moving average of squared gradients — captures the magnitude, enabling per-parameter learning rate scaling.

Strengths of AdamW

- **Battle-tested reliability:** AdamW is the default optimizer for fine-tuning virtually every major LLM, including GPT-3/4, BERT, T5, and LLaMA variants.
- **Stable convergence:** The adaptive learning rates and bias correction make it robust across different model architectures.

- **Correct regularization:** Decoupled weight decay prevents underfitting in dense layers while not over-penalizing sparse parameters.
- **Wide ecosystem support:** Available in PyTorch, JAX/Optax, HuggingFace, and every major ML framework with sensible defaults.

Weaknesses of AdamW

- **Memory overhead:** Requires storing two full copies of model parameters as optimizer states (m and v), effectively tripling the memory footprint of a naive SGD run. For a 70B model, this can mean >400GB of optimizer state alone.
- **Not ideal for massive batches:** At very large batch sizes (tens of thousands), AdamW can become unstable without careful learning rate tuning.
- **Hyperparameter sensitivity:** While defaults work well for fine-tuning, pre-training large models from scratch requires careful LR schedule design (usually cosine annealing with warmup).

When to Use AdamW

Best Use Cases for AdamW

✓ Fine-tuning pre-trained LLMs (RLHF, instruction tuning, domain adaptation) ✓ Training small to medium-scale models (<7B parameters) from scratch ✓ Research experimentation where ease of use and reliability matter ✓ Any task where memory is not the primary constraint

Adafactor Optimizer

As LLMs grew from millions to billions of parameters, the memory cost of AdamW became a serious bottleneck. **Adafactor** was introduced by Shazeer & Stern (Google Brain, 2018) with a radical question: *do we really need to store a separate second-moment estimate for every single parameter?*

Their insight was brilliant — for a weight matrix W of shape $(n \times m)$, instead of storing a full $n \times m$ variance matrix, you can **factorize** it into row and column statistics. This reduces memory from $O(nm)$ to $O(n + m)$ — a massive saving for large embedding and attention matrices.

How Adafactor Works

Adafactor uses a factored second-moment estimate:

- **Row factor R :** An n -dimensional vector tracking per-row gradient statistics.
- **Column factor C :** An m -dimensional vector tracking per-column gradient statistics.
- **Approximation:** The full second moment is approximated as $(R \cdot C^T) / (1^T \cdot C)$, normalized to match the true variance.

Strengths of Adafactor

- **Dramatic memory savings:** Eliminates the first moment entirely (or makes it optional) and factorizes the second moment. Training a 540B model becomes feasible where AdamW would OOM.
- **No learning rate schedule required:** The built-in adaptive step size means one fewer critical hyperparameter to tune — a major advantage when pre-training runs take weeks.
- **Scales to trillion-parameter models:** Google used Adafactor to train PaLM (540B), T5 (11B), and many other large-scale models. It is the optimizer of choice at the frontier.
- **Numerically stable:** Includes clipping and scaling mechanisms that prevent runaway updates even in deep, wide transformers.

Weaknesses of Adafactor

- **Can be unstable for fine-tuning:** The factored approximation and no-momentum setup make Adafactor trickier to use for fine-tuning tasks — AdamW is generally preferred here.
- **Slower initial convergence:** Without momentum (first moment), the updates can be noisier in early training, requiring more steps to reach the same loss as AdamW.
- **Less intuitive configuration:** The adaptive LR schedule can behave unexpectedly for custom training loops, especially when resuming from checkpoints.

When to Use Adafactor

Best Use Cases for Adafactor

✓ Pre-training very large LLMs (>10B parameters) from scratch ✓ Memory-constrained environments (limited GPU VRAM, academic compute) ✓ Research into efficient training at scale ✓ Training on TPUs (it was designed and optimized for this hardware) ✗ Avoid for fine-tuning — AdamW is almost always better

Introduction to Gradient Clipping

The Problem of Exploding Gradients

Training deep neural networks is often compared to navigating a treacherous mountain trail. One of the most perilous obstacles encountered is the **exploding gradients problem**, which can derail the optimization process. In deep models, gradients are computed and propagated backward during training to update model parameters. However, in some cases, these gradients can become excessively large, particularly in **very deep networks** or when working with **long sequences** (as seen in recurrent neural networks). This phenomenon leads to:

Parameter updates of extreme magnitude, causing the model to diverge.

Instability in training, where loss functions fail to converge.

The root cause of this problem lies in the mathematical operations performed during backpropagation. Specifically, the repeated application of derivatives across layers can lead to the exponential growth of gradients.

What is Gradient Clipping?

Gradient clipping is a technique designed to address this challenge by constraining the gradients during backpropagation to a predefined range. By doing so, it ensures that no single gradient update overwhelms the optimization process, thereby **stabilizing training**.

How Does Gradient Clipping Work?

At a high level, gradient clipping adjusts gradients when their magnitude exceeds a specific threshold. This adjustment can be achieved in various ways, which we will explore in subsequent sections. The result is a training process that is more robust, even for **very deep models** or tasks requiring long-term dependencies.

Why Gradient Clipping Matters

The benefits of gradient clipping extend beyond just preventing exploding gradients:

Improved Training Stability: It helps models train smoothly, especially in **Recurrent Neural Networks (RNNs)** or **Transformer-based architectures**.

Better Generalization: By preventing over-aggressive updates, it can indirectly lead to improved generalization performance.

Compatibility with Optimization Algorithms: Gradient clipping integrates seamlessly with optimizers like **Adam**, **RMSProp**, and **SGD**.

Let's dive into the **Methods of Gradient Clipping**, where I'll explain the different approaches and provide the mathematical foundations. Each method is tailored to handle exploding gradients in a slightly different way, offering flexibility depending on the specific use case.

Real-World Applications

Gradient clipping is commonly used in:

Language Models: Models like **GPT** and **BERT**, which handle long-text sequences and require stable training.

Image Recognition Tasks: Deep CNNs, where gradients can grow exponentially with depth.

Time-Series and Sequential Data: RNNs, LSTMs, and GRUs for applications like stock price prediction or speech recognition.

Reinforcement Learning: Environments where instability in gradients can result in erratic policy updates.

Learning Rate Schedulers

Introduction to Learning Rate Schedulers

When training **neural networks**, one of the most critical hyperparameters to tune is the **learning rate (LR)**. The learning rate determines how much the model weights are updated in response to the **gradient** of the loss function during backpropagation. While a high learning rate might cause the training process to overshoot the optimal parameters, a low learning rate can make the process frustratingly slow or get the model stuck in suboptimal local minima.

Learning Rate Schedulers.

A **learning rate scheduler** dynamically adjusts the learning rate during training, offering a systematic way to balance the **trade-off between convergence speed and stability**. Instead of manually tuning the learning rate, schedulers automate its adjustment based on a **predefined strategy** or the **model's performance metrics**, enhancing the efficiency and performance of the training process.

Why Use a Learning Rate Scheduler?

Improved Convergence: Schedulers help avoid overshooting or stagnation during optimization, ensuring the model finds better minima faster.

Stability: By reducing the learning rate as training progresses, schedulers help stabilize training when the model approaches optimal weights.

Generalization: Careful modulation of the learning rate often leads to better generalization, reducing the risk of overfitting.

Automated Fine-Tuning: Schedulers reduce the need for manual intervention, saving time and computational resources.

Key Considerations

When choosing or designing a learning rate scheduler, some **key considerations** include:

The **initial learning rate** and how quickly it should decay.

The **nature of the dataset and model**, such as whether the model is prone to overfitting.

The **training duration**, which can influence whether aggressive or conservative scheduling is needed.

Mixed Precision Training in LLMs:

In the realm of **Large Language Model (LLM)** training, **efficiency and scalability** are as important as accuracy.

Training models like LLAMA 3 with **hundreds of billions of parameters** can cost millions in compute if not optimized carefully.

One key enabler of this efficiency revolution is **Mixed Precision Training**.

In this blog, we will dive into:

What is Mixed Precision Training?

Differences between **FP16**, **BF16**, and **FP8**

How it impacts memory, speed, and convergence

what is Mixed Precision Training?

Mixed Precision Training is the technique of **using multiple numerical data types (precisions) in the same training process** to achieve:

Lower memory footprint

Higher computational speed

While maintaining **model convergence and accuracy**

Instead of using the traditional **FP32 (32-bit float)** everywhere, mixed precision strategically uses:

FP16 (16-bit float) or BF16 (Brain Float 16) for most computations and activations.

FP32 for critical operations like loss scaling, optimizer states, and master weights.

Key Techniques in Mixed Precision Training

Loss Scaling (Static / Dynamic):

Scales up the loss during backward pass to avoid underflows when using FP16/BF16.

Automatic in frameworks like AMP (Automatic Mixed Precision).

2. Master Weights in FP32:

Maintains a master copy of weights in FP32 for stability.

3. Optimizer State in FP32:

Keeps optimizer (like Adam) states in FP32 to prevent precision loss.

What is Quantization in LLM?

Artificial intelligence has rapidly evolved from an academic pursuit to a core business tool. Companies now deeply want to integrate models like ChatGPT into their own secure systems. This desire stems from a non-negotiable need for data privacy, ensuring sensitive information remains entirely on internal networks. Yet the monumental size and computational demands of these models present a massive deployment hurdle. They demand resources often beyond what's readily available.

Then there's quantization in LLM, a powerful technique that reduces the precision of model parameters. This decreases memory usage and improves inference speed. This article will explore the concept of what is quantization in LLMs in the context of LLMs, tracing its history, detailing various types, examining size benefits, understanding perplexity as a performance metric, discussing accuracy loss, and highlighting how AI model quantization makes private LLM hosting viable.

Historical Context of Quantization

Origins of Quantization

Quantization as a concept has deep roots in digital signal processing (DSP) and data compression techniques. Initially, it aimed to convert continuous signals into a format suitable for computer storage and processing. The earliest examples of quantization can be traced back to the development of pulse-code modulation (PCM) in the 1930s, in which analog signals were represented in digital form by sampling and quantizing their amplitudes. As noted in the literature, "Quantization forms the core of essentially all lossy compression algorithms."

Evolution in AI

Then deep learning arrived. The 2010s saw neural networks growing not just in complexity but in sheer size. Their computational and memory demands became increasingly profound. Researchers noticed those old DSP concepts. Pioneering work by people like Courbariaux and Rastegari around 2016 showed us something remarkable. We could actually use low-precision weights inside neural networks. They can be made smaller and faster without completely breaking their performance.

Adoption in LLMs

The transition from traditional models to LLMs, with their billions of parameters, highlighted the increasing need for Large Language Model quantization. When models like Llama exploded in size to billions of parameters, the theoretical became an urgent necessity. Quantization was no longer just an interesting optimization. It was the only way to deploy these behemoths on accessible hardware realistically, especially for teams planning a scalable MCP server deployment for enterprise AI systems.

Current Trends

The field is advancing at a very fast pace right now. We're seeing techniques like post-training quantization gain traction, promising not just smaller models, but smarter, more accurate quantized models. The focus has fully shifted to preserving quality, not just compressing size. It's a fascinating evolution to watch.

Comprehensive Overview of Quantization Formats

The choice of quantization format significantly impacts model performance, memory usage, and computational efficiency. Here's an in-depth look at the most important formats used in quantization in machine learning workflows:

INT8 (8-bit Integer)

INT8 quantization is the trusted workhorse for production systems. It cuts the model size by seventy-five percent compared to FP32. You typically see less than a one percent accuracy drop, which is just remarkable. Through studies, scholars found out that a massive 70B parameter model that needed 280GB of memory suddenly becomes a much more manageable 70GB. The hardware support across different vendors is also excellent.

Key characteristics:

Memory reduction: 4x compared to FP32, 2x compared to FP16

Minimal accuracy loss (typically <1%)

Excellent hardware support across NVIDIA, AMD, and Intel GPUs

Industry standard for production deployments in Large Language Model quantization

FP16 (16-bit Floating Point)

FP16, or 16-bit floating point, offers a nice middle ground. It provides a two times reduction in memory and is a common baseline for inference. Modern GPUs handle these calculations natively with their Tensor Cores, which gives you a nice speed boost almost for free.

INT4 (4-bit Integer)

INT4 quantization provides aggressive 8x memory reduction compared to FP32, making large models accessible on consumer hardware. However, this comes with greater accuracy trade-offs. Several methods have emerged to optimize INT4 quantization, supporting the broader goals of AI model optimization techniques used in deployment.

FP4 (4-bit Floating Point) and NF4 (4-bit NormalFloat)

FP4 represents a newer approach that uses floating-point representation at 4-bit precision. Compared to integer quantization, FP4 handles bell-shaped and long-tail distributions more flexibly. This is particularly advantageous for neural network weights used in quantization in neural networks.

NF4 (NormalFloat 4-bit)

NF4 uses information-theoretically optimal quantization, normalizing weights to fall within a specific range for more efficient representation. Key features include:

Assuming weights follow a Gaussian distribution

Using lookup tables for efficient dequantization

Being integrated with bitsandbytes library for easy deployment

Enabling fine-tuning of 65B+ models on single consumer GPUs

Combining with Double Quantization (DQ) for additional 0.4 bits per parameter savings.

Recent developments

NVIDIA's Blackwell architecture, powering the RTX PRO 6000 and B200, now integrates native FP4 tensor core support. This hardware advancement hints at potentially doubling performance compared to FP8 operations. Real-world benchmarks already reflect this, with FP4 achieving around 1.32 times the speed of FP8 on the same setup, all while preserving competitive accuracy levels. It's a promising step for efficient inference in quantization in machine learning.

FP8 (8-bit Floating Point)

FP8 has emerged as a sweet spot for many production deployments, offering near-lossless accuracy with 2x memory reduction. Modern data center GPUs (H100, H200, B200) include native FP8 tensor cores that deliver 2x performance improvement compared to FP16. Many recent models are trained with FP8 support built in, enabling near-lossless quantization and supporting broader AI model quantization workflows.

Real-world impact:

Meta's Llama 3.3-70B with FP8: 99%+ quality recovery, 30% latency reduction, 50% throughput improvement

Llama 3.2-90B with FP8: 10% latency reduction using half the number of GPUs

FP6 (6-bit Floating Point) and MXFP6

FP6 (6-bit floating point) represents an emerging middle ground between FP4 and FP8. Recent research with microscaling (MX) formats shows MXFP6 can offer better accuracy than MXFP4 while maintaining similar computational efficiency on certain hardware architectures.

Studies on large models (70B+ parameters) show mixed-precision strategies combining MXFP4 and MXFP6 can provide better accuracy than pure MXFP4 without sacrificing computational efficiency, making it increasingly relevant for AI model optimization techniques.

Sub-4-bit Quantization (2-bit and 3-bit)

While experimental, sub-4-bit quantization represents the frontier of extreme compression:

3-bit quantization: Can achieve 10-11x compression with acceptable accuracy for certain use cases

2-bit quantization: Achieves 16x compression but with significant accuracy degradation

HQQ (Half-Quadratic Quantization): A calibration-free method that can quantize models to 2-bit in minutes (vs. hours for other methods)

Research shows that 70B models are less affected by aggressive quantization than smaller models, with extremely low quantization of 70B models remaining somewhat usable. Whereas 8B models at similar bit widths approach random noise thresholds commonly referenced in quantization in neural networks.

Learn more about our AI development services to accelerate intelligent business solutions.

Mixed-Precision and Format-Specific Strategies

Advanced deployments often use mixed-precision approaches:

Per-layer quantization: Different layers use different precisions based on sensitivity analysis

Per-channel quantization: Different channels within layers can use different scales

GGUF format: Optimized for CPU and Apple Silicon, supporting flexible per-layer quantization (Q4_K_M, Q5_K_M variants)

EXL2 format: Optimized for low-latency inference with customizable bit allocations

Bit-width Considerations

Choosing the appropriate bit-width is a core part of the quantization strategy. This decision directly shapes both model performance and resource demands. Reducing the bit-width dramatically shrinks model size, sure, but it often introduces quantization noise that can degrade accuracy. That's why finding the right equilibrium between size and performance is absolutely vital for successful deployment using LLM quantization techniques.

General guidelines:

8-bit (INT8/FP8): Production-ready, minimal accuracy loss, 4x memory reduction

4-bit (INT4/NF4/FP4): Aggressive compression, 8x memory reduction, acceptable accuracy for most use cases

3-bit: Experimental, 10x+ compression, noticeable quality degradation

2-bit: Research-level, extreme compression, significant accuracy loss

Size Benefits of Quantization

Quantization offers significant size benefits for deploying Large Language Models, particularly in scenarios where computational resources are limited, and model compression in deep learning is essential.

Real-world Applications and Cost Savings

Organizations are leveraging quantized models across various applications using modern AI model quantization approaches and scalable MCP server development architectures:

Customer Service Chatbots: Certain open-source small language models can generate responses with quality competitive to OpenAI LLMs, provide comparable yet more reliable latency performance, and reduce costs by up to 29%.

Enterprise Private Hosting: A startup running millions of daily chatbot interactions can save 50–70% by switching from per-token cloud pricing to GPU-leased instances.

Infrastructure: 4x RTX 5090 or 1x RTX 6000 Pro Blackwell configuration.

Enables: Complete data privacy, no external API dependencies.

ROI: 9-12 months for hardware investment vs. API costs.

Knowledge Distillation for LLMs:

Want smaller, faster AI models without losing performance? Knowledge distillation is the answer. It's a method for training a smaller "student" model to mimic a larger "teacher" model, making AI more efficient and cost-effective.

Why It Matters:

Cuts Model Size: Reduces size by up to 40% while retaining 95–97% of performance.

Speeds Up Processing: Smaller models run faster and save energy.

Works Anywhere: Enables deployment on devices with limited computing power, like smartphones.

Key Techniques:

Teacher-Student Framework: The student learns from the teacher's outputs and reasoning.

Loss Functions: Uses soft predictions and advanced metrics like KL divergence to guide learning.

Layer-Level Transfer: Aligns the student's internal layers with the teacher's for better feature learning.

Advanced Methods: Includes multi-stage distillation, quantization, and hybrid approaches like QLoRA for maximum efficiency.

Quick Example:

DistilBERT: 40% smaller than BERT but retains 97% of its performance.

This guide breaks down how to set up distillation pipelines, optimize for specific hardware, and measure results.

Core Knowledge Distillation Techniques

Teacher-Student Model Setup

At the heart of knowledge distillation lies the teacher-student framework. Here, a large, complex "teacher" model passes its learned expertise to a smaller, more efficient "student" model. The teacher, trained on extensive datasets, demonstrates a deep understanding of the task, while the student learns to replicate the teacher's predictions and behavior .

Instead of relying on hard labels (e.g., correct/incorrect answers), the student learns from the teacher's **soft predictions** - probability distributions that reveal the teacher's confidence across various outcomes. These soft predictions offer more nuanced insights, reflecting the teacher's internal reasoning processes.

The size difference between teacher and student models can be substantial. For instance, [GPT-3](#) boasts 175 billion parameters and was trained on 570 GB of text [\[6\]](#). Meanwhile, distilled models aim to be much smaller while retaining much of the teacher's capabilities.

This process leverages the principle of universal approximation, which suggests that neural networks, with enough training data and the right architecture, can approximate any function to a high degree of accuracy [\[3\]](#).

Specialized loss functions play a critical role in ensuring the effective transfer of knowledge, as explored in the next section.

LLM Compression Techniques

Large language models (LLMs) without quantization demand significant GPU memory, greatly increasing deployment costs. This overview covers model compression techniques for LLMs and explores the LLM serving engines that support these quantized methods.

Pruning

Pruning is removing connection from the neural network. We can remove the connections in two ways:

Structured Pruning: All the incoming and outgoing connections for the selected neurons are removed.

Unstructured Pruning: Connection are removed based on the selection logic. Some neuron can have all the existing connections and some neuron might not have any connection and any number in between.

Selection logic can be as simple as checking the magnitude of the weights and removing the weights with magnitude close to zero.

Performance of unstructured pruning is generally better than structured pruning. We can also do more aggressive pruning with unstructured pruning while keeping the model performance.

Press enter or click to view image in full sizeSparse models will require less memory to host and certain level of pruning does not impact the test performance.

Press enter or click to view image in full size

We need to implement sparse inference kernels to provide both memory reduction and acceleration of sparse models. This is why pruning is not that much popular in LLMs compared to quantization.

Distillation

Distillation generally involves using a larger pre-trained model's embedding vector as target to train a smaller model.

Distilled models most of the time performs better than if we just train the model directly on the train data.

Quantization

Model weights are generally stored in float16 format. Quantization is converting them from high precision format (float32, float16) to low precision format (like int8, int4).

LLMs are generally over parameterised and quantization acts as regularization, thus model performance does not degrade drastically.

Quantization also makes sense for real world edge application (personal assistant), where LLMs are more memory bound than compute bound.

All the quantization techniques can be grouped into two categories.

Quantization aware training: The model is trained during quantization process, thus these methods produces more accurate models.

Post training quantization: The model is directly quantized without further training. These models generally less accurate than the models produced by quantization aware training but it require very less compute to create the models.

Because LLMs have large number of parameters, doing quantization aware training is generally expensive. But because they are over-parameterized even with post training quantization the performance drop is very minimal.